Smart Page Replacement Algorithm

Project Report submitted for the partial fulfillment of grade

for the course

Operating Systems And Systems Programming

(25SC2104E)

in

A.Y. 2026-2027

Submitted by

GOUTHAM VIKAS PARIMI (2520090096)

Branch: CSIT

Team Number: 20

Section: 10

Under the guidance of

Raghupathi

Professor, CS(IT)



Hyderabad-500090, Telangana, India.

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY



DECLARATION

The Project Report entitled **"Smart Page Replacement Algorithm"** is a record of bonafide work of **GOUTHAM VIKAS PARIMI – 2520090096**, submitted in partial fulfillment of the requirements of the course Operating Systems and Systems Programming (25SC2104E) for the award of B.Tech in Computer Science and Information Technology at K L (Deemed to be) University. The results embodied in this report have not been copied from any other department/University/Institute.

GOUTHAM VIKAS PARIMI 2520090096

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY



CERTIFICATE

This is to certify that the project report entitled **"Smart Page Replacement Algorithm"** is a bonafide work done and submitted by **Goutham Vikas Parimi – 2520090096** in partial fulfillment of the requirements for the course Operating Systems and Systems Programming (25SC2104E), Bachelor of Technology in the Department of Computer Science and Information Technology, K L (Deemed to be University), during the academic year **2026-2027**.

Signature of the Guide

Signature of the Course Coordinator

Signature of the HOD

ACKNOWLEDGEMENT

The success of this project would not have been possible without the timely help and guidance rendered by many people. I wish to express my sincere thanks to all those who have assisted me in one way or the other in the completion of this project.

My greatest appreciation goes to my guide, Raghupathi, Professor, Department of Computer Science and Information Technology, for his tremendous support, encouragement and guidance, which cannot be expressed in words.

I express my gratitude to the Head of the Department for providing me with adequate facilities, ways and means by which I was able to complete this project.

I thank all the members of the teaching and non-teaching staff who have assisted me directly or indirectly in the successful completion of this project. Finally, I sincerely thank my parents, friends and classmates for their kind help and cooperation during my work.

GOUTHAM VIKAS PARIMI 2520090096

ABSTRACT

The Smart Page Replacement Algorithm is designed to improve memory management in operating systems by reducing page faults and utilizing available memory more efficiently. When the physical memory becomes full and a new page is requested, the operating system must select an existing page for replacement. Traditional algorithms such as First-In, First-Out (FIFO) and Least Recently Used (LRU) have certain limitations in handling different memory access patterns. The proposed Smart Page Replacement Algorithm aims to overcome these limitations by considering factors such as page access frequency, recency of use, and page usage history to identify the least useful page for replacement. The system simulates memory access requests and compares the performance of FIFO, LRU, and the proposed smart algorithm using metrics such as page faults, page hits, hit ratio, and execution time. A visualization interface can be developed to display memory frames and replacement decisions, making the working process easy to understand. The experimental results will help evaluate the effectiveness of the proposed approach under different workloads. This project aims to provide an efficient and user-friendly solution for understanding and

improving page replacement strategies in operating systems. The proposed algorithm’s performance will be validated through experimental comparison rather than assuming it always outperforms existing methods.

CHAPTER 1: INTRODUCTION

Memory management is one of the most important functions of an operating system. When a program runs, its required pages are loaded into Random Access Memory (RAM) for faster access. Since RAM has limited capacity, the operating system must remove an existing page when a new page is requested and all available memory frames are occupied. This process is known as page replacement. Traditional page replacement algorithms, such as First-In, First-Out (FIFO) and Least Recently Used (LRU), use different strategies to select a page for removal. However, their performance depends on the memory access pattern of a program. The Smart Page Replacement Algorithm aims to improve page replacement decisions by considering factors such as page access frequency and recency of use. This project implements a web-based simulator that visualizes memory operations and compares FIFO, LRU, and a customizable Smart algorithm using performance metrics such as page faults, page hits, and hit ratio.

CHAPTER 2: OBJECTIVES

The objectives of the Smart Page Replacement Algorithm project are to develop an interactive web-based application that demonstrates and evaluates different page replacement techniques used in operating systems. The project aims to implement and compare First-In, First-Out (FIFO), Least Recently Used (LRU), and the proposed Smart Page Replacement Algorithm to understand their behavior under different memory access patterns. Another objective is to design a Smart algorithm that considers page access frequency and recency to select suitable pages for replacement. The system also aims to visualize RAM frame allocation, page hits, page faults, and replacement decisions, making memory management concepts easier to understand. Furthermore, the project focuses on calculating and comparing performance metrics such as page-fault count, page-hit count, hit ratio, fault ratio, and execution time. Finally, it aims to provide a user-friendly interface with configurable inputs, graphical performance analysis, and CSV export functionality to support experimentation and help identify how different page replacement strategies perform under various workloads.

CHAPTER 3: PROBLEM STATEMENT

Traditional page replacement algorithms may remove useful pages from memory, resulting in additional page faults and increased memory access overhead. FIFO selects pages based on their arrival order, while LRU considers only the time since the last access. Neither strategy directly combines both access frequency and recency in its basic form. Therefore, there is a need to develop and evaluate an improved page replacement strategy that considers multiple page usage characteristics. The project addresses this problem by implementing a configurable Smart Page Replacement Algorithm and comparing its performance against FIFO and LRU under identical workloads.

CHAPTER 4: EXISTING SOLUTION

Existing page replacement methods use predefined rules to select a page for removal when memory becomes full.

FIFO (First-In, First-Out): Replaces the page that entered memory first. It is simple to implement but does not consider whether the page is frequently accessed.

LRU (Least Recently Used): Replaces the page that has not been accessed for the longest time. It considers recent usage but does not directly account for cumulative access frequency.

These algorithms provide useful baseline strategies, but their effectiveness varies according to the page reference sequence. The proposed project uses them as benchmarks to evaluate a multi-factor replacement strategy.

CHAPTER 5: PROPOSED SOLUTION

The proposed system is a browser-based Smart Page Replacement Simulator developed using HTML, CSS, and JavaScript. It allows users to enter a page reference sequence and select the number of available RAM frames. The simulator executes FIFO, LRU, and Smart replacement policies on the same input and displays their results. The Smart algorithm assigns a score to each resident page based on its access frequency and recency. When memory is full, the page with the lowest score is selected for replacement. Users can adjust the frequency and recency weights to experiment with different strategies. The application also provides animated RAM visualization, performance comparison charts, a results table, and CSV export functionality. The objective is to evaluate whether the Smart strategy reduces page faults for different workloads rather than assuming that it always outperforms traditional algorithms.

CHAPTER 6: TECHNOLOGIES USED

The Smart Page Replacement Algorithm project is developed using HTML, CSS, and JavaScript. HTML5 is used to create the structure of the web application, including input fields, buttons, RAM frame displays, and result tables. CSS3 is used to design an attractive, responsive, and user-friendly interface with suitable colors, layouts, and visual effects. JavaScript is used to implement the FIFO, LRU, and Smart Page Replacement algorithms, process page requests, manage simulated memory frames, and calculate page hits, page faults, hit ratios, and fault ratios. The Chart.js library is used to represent algorithm performance through graphical charts, making it easier to compare results. A modern web browser, such as Google Chrome or Microsoft Edge, is used to run the application. Visual Studio Code can be used as the code editor for development and testing, while Git and GitHub can be used for version control and source code management. The CSV export feature allows users to download performance comparison results for further analysis and documentation.

CHAPTER 7: SYSTEM DESIGN

The system design of the Smart Page Replacement Algorithm consists of several interconnected modules that work together to simulate memory management and compare different page replacement techniques. The system begins with the user interface, where users enter the page reference sequence, specify the number of RAM frames, and configure the frequency weights for the Smart algorithm. The input processing module validates the entered data and passes the page requests to the page replacement engine. This engine implements three algorithms: First-In, First-Out (FIFO), Least Recently Used (LRU), and the proposed Smart Page Replacement Algorithm. FIFO replaces the page that entered memory first, while LRU replaces the page that has not been used for the longest time. The Smart algorithm evaluates the frequency and recency of page accesses to select a suitable page for replacement. The simulation module processes each page request and updates the memory frames, recording page hits, page faults, and replacement decisions. The performance analysis module calculates the total page hits, page faults, hit ratio, fault ratio, and execution time for each algorithm. Finally, the visualization module displays the RAM frame changes,

performance comparison charts, and results table on the web interface. Users can also export the results as a CSV file for further analysis. This modular system design makes the application interactive, easy to understand, and suitable for demonstrating and evaluating page replacement strategies in operating systems.

CHAPTER 8: FLOWCHART

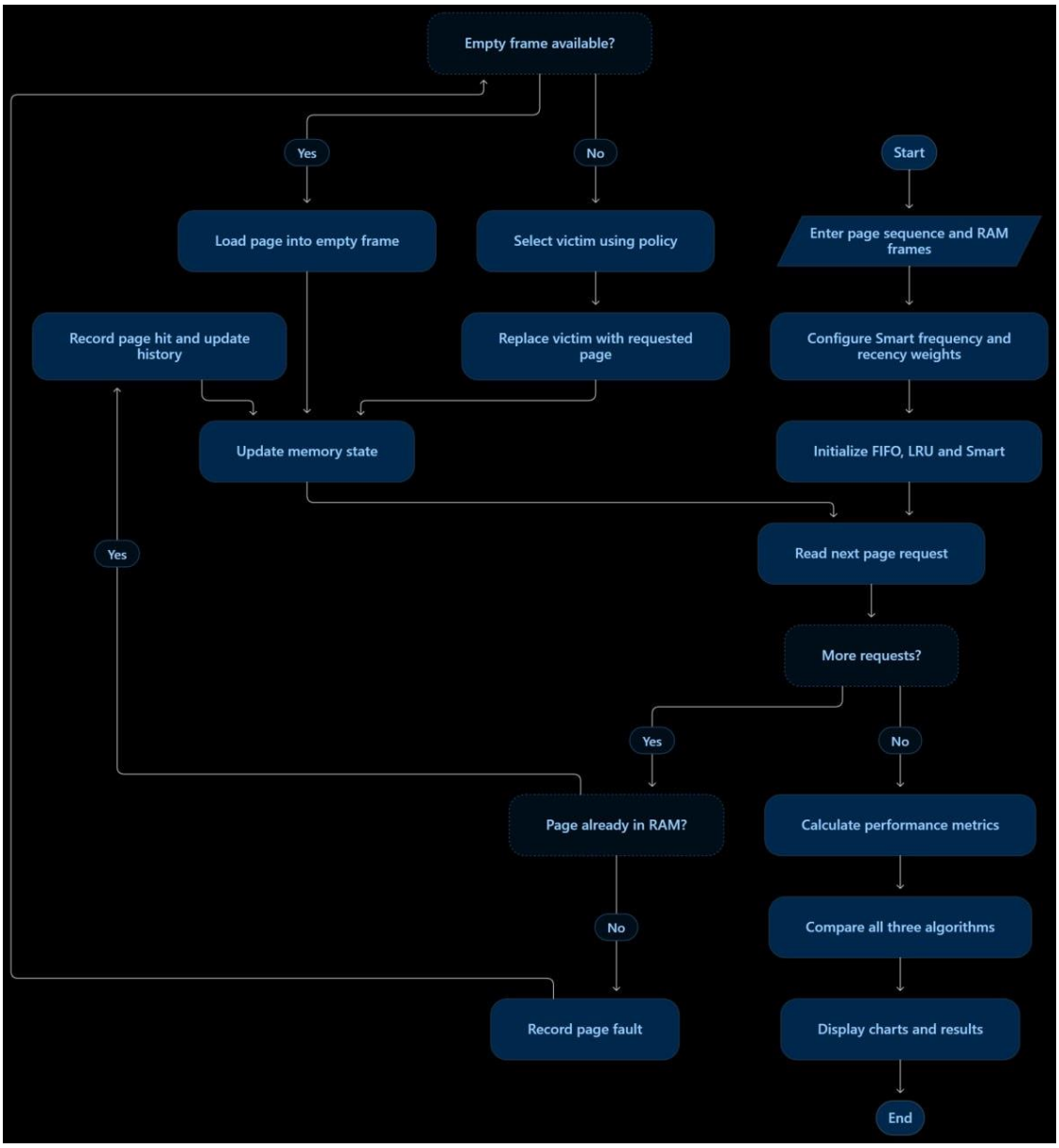


Fig. 1: Flowchart of the Smart Page Replacement Simulator

CHAPTER 9: IMPLEMENTATION APPROACH

The implementation approach of the Smart Page Replacement Algorithm begins with developing a web-based application using HTML, CSS, and JavaScript. HTML is used to create the user interface, including input fields for the page reference sequence, RAM frame count, animation speed, and Smart algorithm configuration. CSS is applied to design a responsive layout with memory frame displays, buttons, charts, and result tables. JavaScript is used to implement the three page replacement algorithms: First-In, First-Out (FIFO), Least Recently Used (LRU), and the proposed Smart Page Replacement Algorithm. Initially, the user enters a sequence of page requests and specifies the number of available memory frames. The application then processes each request and checks whether the requested page is already present in memory. If the page is available, a page hit is recorded; otherwise, a page fault occurs, and the application either loads the page into an empty frame or replaces an existing page according to the selected algorithm. The Smart algorithm considers both page access frequency and recency to calculate replacement scores and select a page for eviction. During simulation, the application records memory changes, page hits, page faults, and execution

time. The Chart.js library is used to display graphical comparisons of algorithm performance, while a results table presents the calculated metrics, including hit ratio and fault ratio. Finally, users can observe the simulation step by step and export the comparison results in CSV format. The implementation is tested using different page reference sequences and memory frame configurations to evaluate the behavior of each algorithm and verify the correctness of the results.

CHAPTER 10: WEB APPLICATION SCREENSHOT

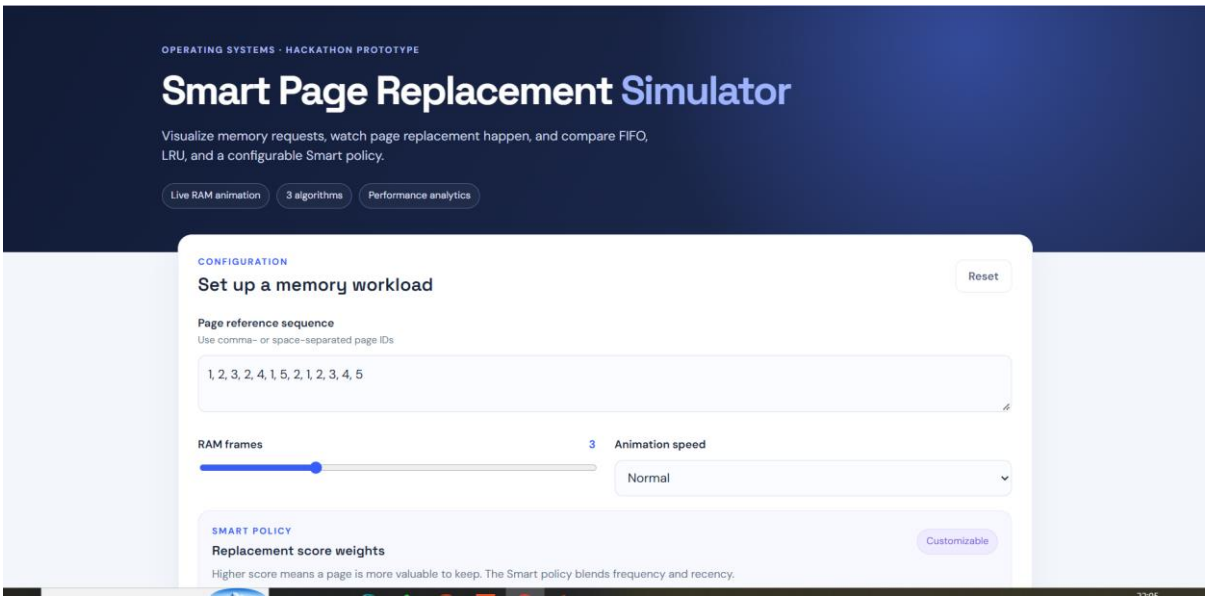


Fig. 2: Home screen of the Smart Page Replacement Simulator

CHAPTER 11: INPUT SCREENSHOTS

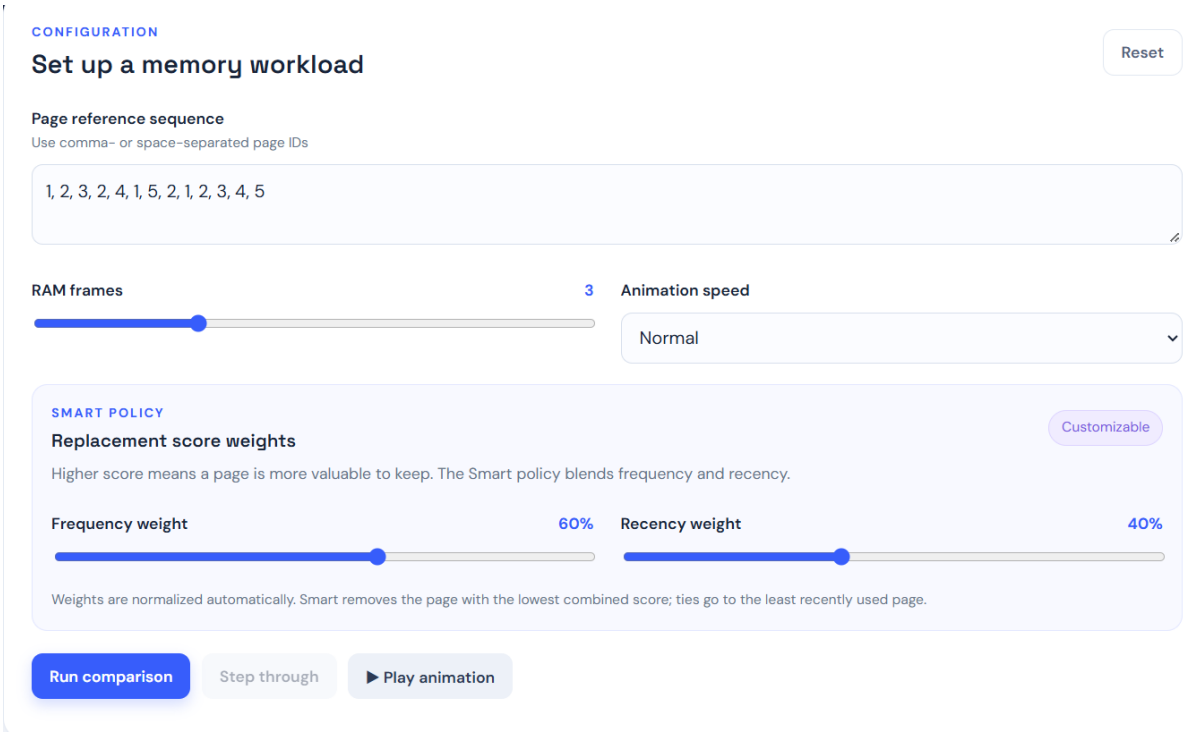


Fig. 3: Input configuration (page reference sequence, RAM frames, animation speed and Smart policy weights)

CHAPTER 12: OUTPUT SCREENSHOTS

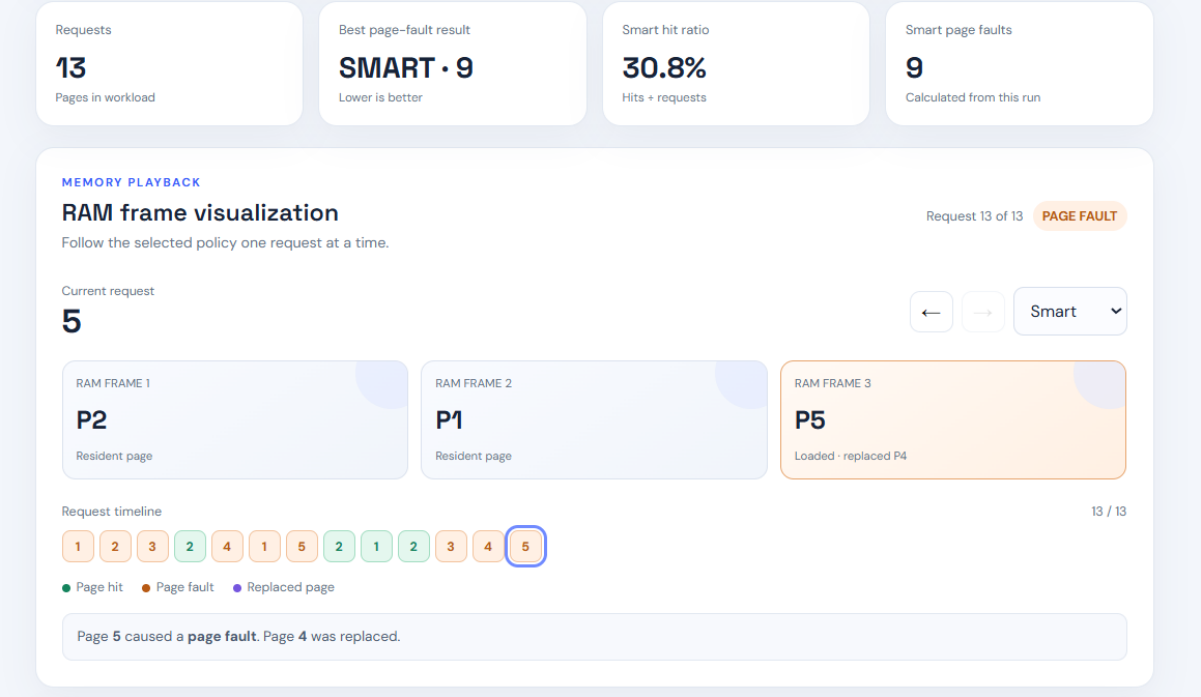


Fig. 4: Output showing RAM frame visualization, request timeline and summary metrics

CHAPTER 13: COMPARISON

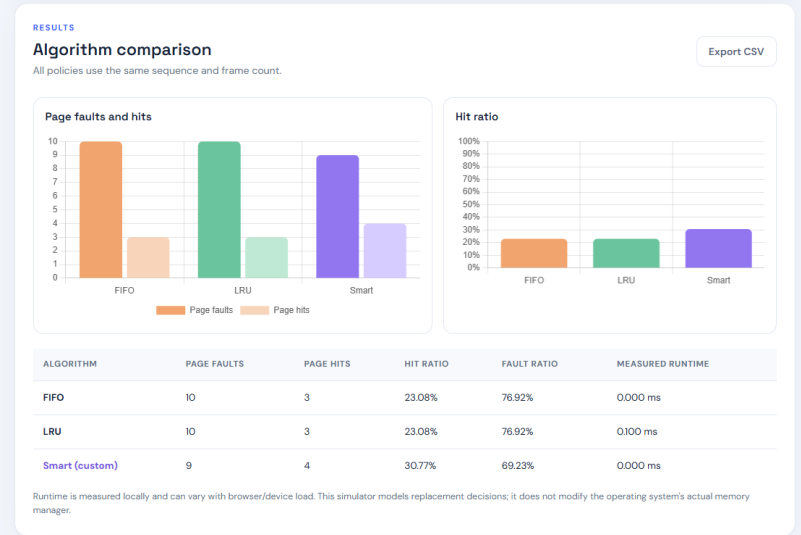


Fig. 5: Algorithm comparison charts and results table

Algorithm	Page Faults	Page Hits	Hit Ratio	Fault Ratio	Runtime
FIFO	10	3	23.08%	76.92%	0.000 ms
LRU	10	3	23.08%	76.92%	0.100 ms
Smart (custom)	9	4	30.77%	69.23%	0.000 ms

Table 1: Results for the sequence 1, 2, 3, 2, 4, 1, 5, 2, 1, 2, 3, 4, 5 with 3 RAM frames

Runtime is measured locally and can vary with browser/device load. This simulator models replacement decisions; it does not modify the operating system's actual memory manager.

CHAPTER 14: IMPACT

The Smart Page Replacement Algorithm project provides an interactive learning environment for understanding operating system memory management. It helps students visualize page faults, page hits, and memory replacement decisions rather than relying only on theoretical explanations. By comparing multiple algorithms on the same workload, the system demonstrates how different replacement policies affect memory performance. The project also introduces practical concepts such as algorithm design, performance measurement, data visualization, and web application development. Its configurable scoring mechanism allows users to experiment with replacement policies and investigate the relationship between access patterns and memory efficiency. Although the prototype does not modify a real operating system, it can serve as an educational foundation for future memory management research.

CHAPTER 15: CONCLUSION

The Smart Page Replacement Algorithm project presents a web-based simulator for understanding and comparing memory replacement strategies. It implements FIFO and LRU as baseline algorithms and introduces a customizable Smart policy that combines page access frequency and recency. The application provides animated RAM frame visualization, step-by-step execution, performance comparison charts, a results table, and CSV export. The system enables users to evaluate the behavior of the three algorithms under different memory workloads. Its effectiveness is assessed using page faults, page hits, hit ratio, fault ratio, and runtime. The project demonstrates how a multi-factor replacement strategy can be designed and experimentally evaluated. Further testing across diverse workloads is necessary to determine when the Smart algorithm offers a practical advantage over traditional approaches.

CHAPTER 16: FUTURE SCOPE

The future scope of the Smart Page Replacement Algorithm project includes several improvements to enhance memory management simulation and algorithm performance. The system can be extended by incorporating additional page replacement algorithms, such as Optimal Page Replacement, Least Frequently Used (LFU), and Clock Page Replacement, to provide a more comprehensive comparison. The Smart algorithm can be improved by using adaptive frequency weights that automatically adjust according to memory access patterns. Machine learning techniques could also be explored to predict future page requests and make more effective replacement decisions. Furthermore, the application can be enhanced with advanced visualizations, real-time performance monitoring, larger datasets, and automated testing across different workloads. Additional features, such as downloadable PDF reports, detailed execution logs, and customizable simulation settings, could make the system more useful for students and researchers. In the future, the project could also be integrated into an operating system simulator to demonstrate memory management concepts in a more realistic environment. These enhancements would help improve the application's

educational value and support further research into efficient page replacement strategies.